Cadence IPs for 2.5G and Gigabit Ethernet MAC

Part Numbers: IP701x

IP Rev: R1p12 - Doc Rev: 1.1

Quick Start Guide

November 6, 2017

Table of Contents

1 Getting Started 6
- 1.1 Documentation Set 6
- 1.2 Workflow-Documentation Map 7

2 Setting Up the Environment 8

3 Installing the IP 9
- 3.1 Directory Structure 9
- 3.2 Supplied Files 10
- 3.3 Creating a Design Configuration 10
  - 3.3.1 Option with Perl TK 11
  - 3.3.2 Option without Perl TK 11

4 Running Sanity Simulation 12
- 4.1 Prerequisites of Running Sanity Simulation 12
- 4.2 Running the Verilog TB Simulation Scripts 12
- 4.3 Checking the Verilog TB Simulation Results 12
- 4.4 Running the Demo TB 12
  - 4.4.1 UVM Register Model 13
  - 4.4.2 Demo TB Documentation 13
- 4.5 Checking the Demo TB Results 13

5 Running Sanity Synthesis 14
- 5.1 Prerequisites of Running Synthesis 14
- 5.2 Running Synthesis Scripts 14
- 5.3 Checking Synthesis Results 14

About this Document

The intended audience of the Quick Start Guide is developers as well as design, verification, and software engineers. This document allows you and your team to easily find the information needed for the current stage of your design development. With this document, you can quickly deploy Cadence IP in simulation or in your target system. This document provides:

- Step-by-step guidance: How to set up the environment, unpack files, and run out-of-the-box sanity checks
- A mapping of design development workflow to the relevant documentation
- Delivery directory structure

This guide is organized as follows:

Section 1 Getting Started on page 6

Section 2 Setting Up the Environment on page 8

Section 3 Installing the IP on page 9

Section 4 Running Sanity Simulation on page 12

Section 5 Running Sanity Synthesis on page 14

Part Numbers

The guide is applicable to the Cadence product part numbers listed in the following table:

Table 1: Part Numbers

Part Number	Product
IP7017	Cadence IP for 2.5G Ethernet MAC with DMA, 1588, TSN/AVB, and PCS
IP7017A	Cadence IP for 2.5G Ethernet MAC with DMA, 1588, TSN/AVB, and PCS for Automotive
IP7016	Cadence IP for 2.5G Ethernet MAC
IP7014	Cadence IP for Cadence IP for Gigabit Ethernet MAC with DMA, 1588, and TSN/AVB
IP7014A	Cadence IP for Gigabit Ethernet MAC with DMA, 1588, and TSN/AVB for Automotive
IP7010	Cadence IP for Gigabit Ethernet MAC

Release Information

Table 2 shows the release information of the Cadence IPs in the table above.

Table 2: Release Information

Item	Description
IP revision	r1p12
Release date	November 22, 2017

References and Related Documents

This guide references the following documents:.

Table 3: References and Related Documents

Reference	Version/Description
Cadence IP for GEM User Guide	Refer to *Table 1* for the name of your IP for the User Guide.

Acronyms and Abbreviations

The following acronyms and abbreviations are used in this guide:

Table 4: Acronyms and Abbreviations

Term	Definition
ATPG	Automatic Test Pattern Generation
APB	Advanced Peripheral Bus
AXI	Advanced eXtensible Interface
DMA	Direct Memory Access
DFT	Design for Test
HDL	Hardware Description Language
RDL	Register Definition Language
RTL	Register Transfer Level
SoC	System on Chip
TB	Testbench
UVC	UVM Verification Component
UVM	Universal Verification Methodology
VIP	Verification IP

Typographical Conventions

The following conventions are used in this guide:

Table 5: Typographical Conventions

Convention	Description
`courier font`	Indicates code

1 Getting Started

The Cadence IPs, as listed in *Table 1 Part Numbers* on page 4, are subsequently also referred to in this document as GEM. GEM is delivered with a comprehensive set of documentation to facilitate integration.

This chapter provides:

- A listing of Cadence technical documentation for GEM
- A workflow-documentation mapping for each stage of your design development

1.1 Documentation Set

Table 6 lists the key documentations for GEM. Please find all the relevant documentation in the documentation directory of the delivered files.

Table 6: IP Documentation Set

Type	Description
Quick Start Guide	How to get GEM up and running by using out-of-the-box installation sanity check. It also includes a workflow-document map that directs you to the relevant documentation.
User Guide	How to configure, set up, use, and verify the GEM core. It also describes the architecture, pre-configuration, pin list, and registers. Where applicable, it may include the Programmer's Section and Application Notes.
Integration Guide	How to integrate GEM into your SoC. It provides details of the deliverables; I/O, clocks, and resets; low-power, synthesis, and physical implementation notes; technology-dependent module replacement; and support notes for Lint, CDC, and STA.
Safety Manual	Relevant for automotive applications only. This manual provides an overview of the features and requirements in safety-related automotive designs according to ISO 26262. It describes any additional system-level hardware or software safety mechanism that may be implemented to achieve the desired system-level Safety Integrity Level.
Release Note	Outlines of the feature enhancement of each subsequent release of GEM.
Errata Notes	Details on known issues. Workarounds are provided, if possible. An optional document, it is included only if there are known and yet to be fixed issues.

1.2 Workflow-Documentation Map

Figure 1 shows a task-based workflow chart for installing the GEM IP.

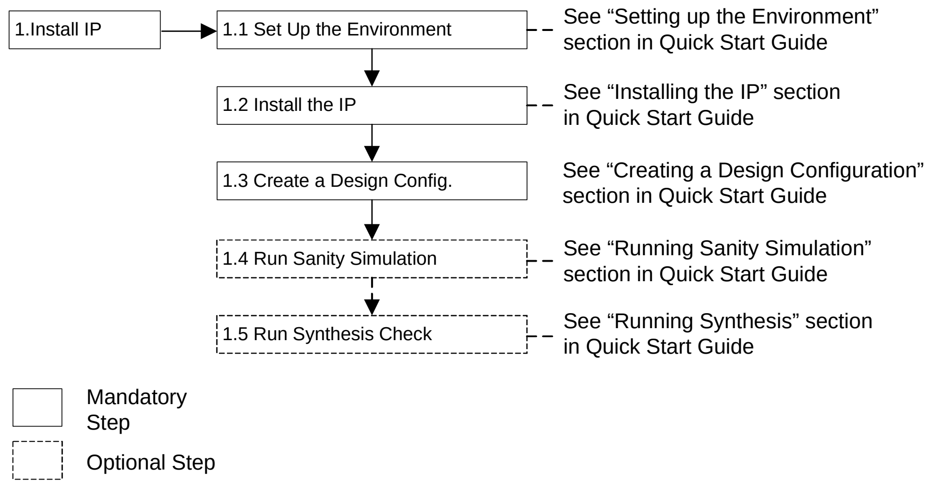

Figure 1: Workflow-Documentation Map

2 Setting Up the Environment

Table 7 lists all the software tools and their versions used by the Cadence delivery environment. This information is provided only to assist you in interpreting the results you may obtain with GEM using your own tools.

However, to run the Verilog TB simulation it is necessary to have installed a Verilog simulator tool such as Cadence Incisive and Perl on your system. To run the Demo TB simulation, it is also necessary to install the VIPCAT library. Please contact your Cadence Application Engineer/sales contact for access to the VIPCAT library.

Table 7: Required Tools

Purpose	Tool	Version
Configuration	Perl	5.6.1 built for i686-linux
Logic Verification	Cadence Incisive® Simulator	15.20 (r020)
VIP catalog for demo testbench	VIPCAT	11.30
Logic Synthesis	Cadence Genus™	16.21
Place and Route	Cadence Innovus™	16.21
Automatic Test Pattern Generation	Cadence Modus™	16.20

Although Cadence has taken all precautions to write the RTL code and the test environment in accordance with the HDL languages standards (Verilog), we cannot guarantee that the user will get the same results as presented in our materials if using tools, or versions of tools, other than those listed in *Table 7*.

3 Installing the IP

GEM is shipped in a single compressed tarball. *Table 8* lists the filenames of the tarball for each part number.

Table 8: Filenames of Tarballs

Part Number	Filename
IP7017	gem_gxl_det0103_r1p12.tar.gz
IP7017A	gem_gxl_det0105_r1p12.tar.gz
IP7016	gem_gxl_det0106_r1p12.tar.gz
IP7014	gem_gxl_det0102_r1p12.tar.gz
IP7014A	gem_gxl_det0104_r1p12.tar.gz
IP7010	gem_gxl_det0100_r1p12.tar.gz

The following describes the installation of the model in a Linux or Unix environment. To install GEM, execute these steps using the tarball's filename (*Table 8*) for your IP delivery:

1. Copy the GEM delivery pack file to your local directory.

   ```
   cp  <filename of tarball>  ./
   ```

   E.g. For IP7017 release 1p12: `cp  gem_gxl_det0103_r1p12.tar.gz  ./`

2. Un-tar the archive

   ```
   tar –zvxf <filename of tarball>
   ```

   E.g. For IP7017 release 1p12: `tar –zvxf gem_gxl_det0103_r1p12.tar.gz`

   to unpack the delivery to a directory named `gem_gxl_det0103_r1p12`

3.1 Directory Structure

The RELEASE_TREE top-level file shows the directory structure. *Table 9* shows the default installation directory by part numbers.

Table 9: Default Installation Directory

Part Number	Installation Directory
IP7017	gem_gxl_det0103_r1p12/
IP7017A	gem_gxl_det0105_r1p12/
IP7016	gem_gxl_det0106_r1p12/
IP7014	gem_gxl_det0102_r1p12/
IP7014A	gem_gxl_det0104_r1p12/
IP7010	gem_gxl_det0100_r1p12/

To see the contents of the delivery, execute:

```
more <installation directory>/RELEASE_TREE
```

E.g. For IP7017 release 1p12: `more gem_gxl_det0103_r1p12/RELEASE_TREE`

Figure 2 shows the delivery structure after you have installed the GEM IP.

```
gem_gxl
|-- dft
|-- doc
|-- func_ver
|-- hdl
|-- models
|-- pwr_intent
|-- software
|-- sta
|-- synth
|-- work
```

Figure 2: Delivery Structure

Note: The delivery directory structure for all Cadence IP Group (IPG) products has a consistent look and feel. Customer delivery subdirectories are intended to accommodate each possible type of design view or data set that could generally apply to Cadence Design IP. Hard and soft IP deliveries each use a unique subset of these directories, with these subsets being common across all IPG business units. Therefore, not all directories are applicable for individual designs hence they are left intentionally empty.

3.2 Supplied Files

Table 10 lists the types of files that comes with each delivery of the GEM IP.

Table 10: Delivery File Details

Directory	Subdirectory/Files	Description
dft		This directory is empty for GEM
doc		See *Table 6 IP Documentation Set* on page 6
func_ver	gatesim	Gate simulation information
	hvm	High-volume manufacturing testbench (TB)
	sanity	Sanity/two TBs (one is a pure Verilog directed TB and the other demonstrates Cadence VIP and test scenarios written in C)
	sva	SystemVerilog assertions
	sdf	Delay back-annotation files
hdl	hdl_src	Unencrypted synthesizable design source
software	CoreDriver	Core driver source code
	Doc	Driver documentation
	README.txt	
	ref_code	Linux driver reference code
	RELEASENOTE.txt	Release history
synth	constraints	Functional and scan mode constraints files
	logs	This directory is empty for GEM
	technology	This directory is empty for GEM
	scripts	Support scrips for synthesis run and analysis
work	Makefile	Make file for running simulation and synthesis scripts
	gem_cfg_builder.pl	Script for creating ancillary files for a particular configuration
	cfg_gen_pkg	Directory containing data associated with previously generated configurations and utilities used by the gem_cfg_builder.pl script

3.3 Creating a Design Configuration

Before GEM can be used, you must create a design configuration matching the user's requirements. GEM has many parameterization options to configure the IP during the compilation stage. This is achieved using Verilog `define compiler directives in an include file called `gem_gxl_defs.v`.

To aid in debug, these configurations are readable through the APB address space starting from address 0x280 (Refer to *Section 11 Registers* of the GEM User Guide for further details).

The release is packaged with a configurator tool called `gem_cfg_builder.pl` to aid the user in creating configuration-specific files needed for compilation and synthesis, and also some other useful files that can help you to understand any LINT/CDC paths that might show in SoC-level quality flows. More information on this tool can be found by launching it with the `-help` argument.

This tool can be launched as follows:

```
cd gem_gxl/work

./gem_cfg_builder.pl -help
```

3.3.1 Option with Perl TK

The first and best option is to launch the included configuration GUI tool that requires a Perl TK package. Offering detailed descriptions of the options available, this GUI-based tool can automatically generate the configuration files needed in order to simulate and synthesize the design with the appropriate options. The main files generated are a file listing, a configuration Verilog defines file, and a synthesis constraints file.

The tool can be launched from the work directory as follows:

```
cd gem_gxl/work

./gem_cfg_builder.pl -gui
```

This tool can save a configuration to a file with a `<user_config_name>.cee` extension. It is recommended the configuration is saved for future reference. Also if this file is provided back to Cadence, some configuration-specific register views can be generated such as IP-XACT, RDL, and register descriptions in RTF format.

3.3.2 Option without Perl TK

If Perl TK is not available, the second option is to create the Verilog defines file manually, and then use the supplied `cfg_builder.pl` tool to generate the remaining configuration-specific files. The delivered database is provided with a few example fixed configurations, such as:

`gem_gxl/work/cfg_gen_pkg/fixed_cfgs/pbuf_3qs_axi.cee.`

This is a AXI multi-queue DMA packet buffer configuration.

1. First use the cfg_builder tool to generate the Verilog defines file for this example configuration

   ```
   cd gem_gxl/work

   gem_cfg_builder.pl -cfg pbuf_3qs_axi
   ```

2. Open “`gem_gxl_defs.v`” file generated in step 1 using a text editor follow the comments for each option update to the desired configuration. The comments should provide sufficient information to aid selection.
3. Save the updated Verilog defines file as `<user_config_name>.v`
4. Import the updated defines file to generate a configuration file in the `.cee` extension format

   ```
   gem_cfg_builder.pl -import <user_config_name>.v
   ```

5. The `<user_config_name>.cee` will now be stored in `./cfg_gen_pkg/imported_cfgs`
6. Run the config builder tool again to generate some important files for simulation and synthesis.

   ```
   gem_cfg_builder.pl -cfg <user_config_name> -gen_synth
   ```

4 Running Sanity Simulation

4.1 Prerequisites of Running Sanity Simulation

Please refer to *Table 7 Required Tools* on page 8 for versions of tools in order to achieve a seamless simulation run.

Note: It is assumed that the Cadence Incisive and Cadence VIPCAT have all been installed and set up with all relevant environment variables for these tools configured in your current shell. Please contact your Cadence Application Engineer/ sales contact for access to the VIPCAT library.

You need to execute the following steps before running the first simulation:

- Set up environment variables
- Generate the configuration-specific files as explained in *Section 3.3 Creating a Design Configuration* on page 10

4.2 Running the Verilog TB Simulation Scripts

This simulation environment uses Perl to translate test scenarios to test stimuli and comparison data. The testbench itself is written in Verilog.

Change to directory `gem_gxl/work`

To run the simulation, use the following command:

```
make run_sim TB=directed CFG=<user_config_name> TST=txrx_soak
```

To run all available test scenarios, use the following command:

```
make run_sim TB=directed
```

4.3 Checking the Verilog TB Simulation Results

The transcript of the simulation is available in a file called irun.log

If successful, then the last few lines of the irun.log file will be similar to the following:

```
                                          txrx_soak    **** PASSED ****
Simulation complete via $finish(1) at time 95881 NS + 0
../func_ver/sanity/vlog_tb/src/common/tb_top.v:2616                #600 $finish;  // wait a bit then finish”
```

4.4 Running the Demo TB

This simulation environment demonstrates the use of VIP and System Verilog to test the IP component.

Change to directory `gem_gxl/work`

The demo TB is supplied with example test scenarios written in C and system Verilog.

To see what test scenarios are available, use the following command:

```
make show_tests TB=uvm_demotb
```

The source for the example C tests is in the directory:

```
func_ver/sanity/demo_tb/verif/uvc_lib/cdn_gem_demo/sve/tests_c/
```

The source for the example UVM tests is in the directory:

```
func_ver/sanity/demo_tb/verif/uvc_lib/cdn_gem_demo/sve/tests/
```

To run the demo TB simulation for the cdn_gem_demo_c_uc_enet_txrx_3pkts_test scenario, use the following command:

```
make run_sim TB=uvm_demotb CFG=pbuf_axi_uvm TST=cdn_gem_demo_c_uc_enet_txrx_3pkts_test
```

4.4.1 UVM Register Model

The UVM register model for this configuration is supplied in:

```
gem_gxl/work/cfg_gen_pkg/fixed_cfgs/reg_models/gem_gxl_pbuf_axi_uvm.sv
```

If you want to run a different configuration you will need to contact Cadence to be supplied the necessary UVM register model system Verilog file.

4.4.2 Demo TB Documentation

The user guide for the demo testbench is located here:

```
func_ver/sanity/demo_tb/doc/Cadence_GEM_GXL_Demo_Testbench_User_Guide.pdf
```

Documentation for C code here:

```
func_ver/sanity/demo_tb/verif/uvc_lib/cdn_gem_demo/docs/docs_doxygen/index.html
```

Documentation for the UVM code available here:

```
func_ver/sanity/demo_tb/verif/uvc_lib/cdn_gem_demo/docs/docs_nd/index.html
```

4.5 Checking the Demo TB Results

The transcript of the demo TB simulation is available in a file called `irun_<test_name>.log`

E.g. `irun_cdn_gem_demo_c_uc_enet_txrx_3pkts_test.log`

If successful then the log file will contain lines as shown in *Figure 3*.

```
------------------------------------------------------------
-  _____  _____  _____  _____   _____  _____  _____  _____  -
- |_   _||  ___||  ___||_   _| |  _  ||  _  ||  ___||  ___| -
-   | |  | |___ | |___   | |   | |_| || |_| || |___ | |___  -
-   | |  |  ___||___  |  | |   |  ___||  _  ||___  ||___  | -
-   | |  | |___  ___| |  | |   | |    | | | | ___| | ___| | -
-   |_|  |_____||_____|  |_|   |_|    |_| |_||_____||_____| -
------------------------------------------------------------
```

Figure 3: Message for a Successful Test

5 Running Sanity Synthesis

This chapter describes how to run out-of-the-box synthesis.

The scripts for synthesis are gathered in the `gem_gxl/synth/scripts` directory and its subdirectories.

There are three main setup files used for the synthesis:

- Setup for paths to RTL sources, output report directories, etc.: `setup_project.csh`
- Additional settings including list of DFT settings, etc.: `project.tcl`
- Settings for the technology libraries (pointers to .lib files): `tech_lib_setup.tcl`

5.1 Prerequisites of Running Synthesis

To run a trial synthesis, it is assumed you have created the configuration specific files. If so, the SDC constraints for the user configuration will have been generated and placed in:

```
gem_gxl/synth/constraints/cfg/<user_config_name>/gem_<user_config_name>.func.sdc
```

Setup file, such as `project.tcl` and `setup_project.csh` needed by the flow are also generated and placed in the directory: `gem_gxl/hdl_qc/cfg/<user_config_name>/`

Please note that the scripts require access to technology libraries referenced by

```
gem_gxl/synth/scripts/tech_lib_setup.tcl
```

An example of the files needed are shown in

```
gem_gxl/synth/scripts/tech_lib_example.tcl
```

5.2 Running Synthesis Scripts

Trial synthesis can be run using the following commands:

```
cd gem_gxl/work

make run_synth CFG=<user_config_name>
```

5.3 Checking Synthesis Results

On completion, the synthesis results should be placed in the directory

```
gem_gxl/synth/default/<user_config_name>/reports/
```

- The file rc.log contains the transcript of the synthesis run
- The file `gem_qor.rpt` summarizes the results of the synthesis run

Change Log

Revision	Date	Description
1.0	June 15, 2017	First release for R1p11
1.1	November 6, 2017	• Aligned document structure and style with updated guidelines. • Updated content for R1p12, e.g. new directory paths